

Implementação do jogo "Genius" na FPGA DE2

Arthur Oliveira Amorim, 25/2034979

Iuri Costa Cavalcante, 24/1009931

João Pedro Nunes Rodrigues, 25/2001181

Grupo C3

¹Dep. Ciência da Computação – Universidade de Brasília (UnB)

CIC0231 - Laboratório de Circuitos Lógicos

July 11, 2026

arthurolimorim@gmail.com, iuri.ccavalcante@gmail.com, jpnrodrigues2007@gmail.com

Abstract. *Implementation of an AND gate using only OR and NOT.*

Resumo. *Implementação de uma porta AND usando apenas OR e NOT.*

1. Introdução

Este projeto final consiste na implementação do jogo *Genius* (também conhecido como *Simon*) na placa de desenvolvimento FPGA DE2. No jogo, uma sequência de cores é apresentada ao jogador, que deve reproduzi-la corretamente pressionando os botões correspondentes; a cada acerto, uma nova cor é adicionada à sequência, e a partida termina quando o jogador comete um erro ou demora demais para responder. O projeto foi descrito em SystemVerilog e desenvolvido no Quartus II, envolvendo a geração pseudoaleatória de cores, o armazenamento da sequência do jogo, o controle do fluxo da partida por meio de uma máquina de estados finita, a contagem de pontos em BCD exibida nos displays de sete segmentos, e a geração de sinais sonoros no buzzer.

1.1. Objetivos

Os objetivos deste projeto são:

- Projetar e implementar, em SystemVerilog, o jogo Genius completo na placa FPGA DE2, integrando lógica combinacional e sequencial de acordo com os requisitos especificados no enunciado do projeto;
- Aplicar conceitos de máquinas de estados finitas para controlar o fluxo do jogo, incluindo os modos de exibição da sequência, espera pela jogada do usuário, verificação de acertos/erros e condição de derrota por tempo excedido;
- Implementar um gerador de números pseudoaleatórios (LFSR) para o sorteio das cores e um módulo de memória para armazenar a sequência gerada ao longo da partida;
- Desenvolver um sistema de contagem de pontos em BCD, com decodificação para os displays de sete segmentos, e um controlador de áudio para os sinais sonoros do jogo;
- Validar o projeto por meio de simulações funcionais e temporais no Quartus II/-ModelSim antes da integração final, e testar o sistema completo fisicamente na placa DE2.

2. Procedimentos de Implementação

2.1. Metodologia e Ferramentas

O projeto foi descrito em SystemVerilog e desenvolvido no Quartus II v.13.0, seguindo uma metodologia incremental: cada subsistema do jogo (pontuação/display de sete segmentos e a máquina de estados com o buzzer) foi primeiro prototipado e simulado isoladamente em projetos Quartus próprios, com testbenches gerados a partir de formas de onda (*Vector Waveform Files*) no ModelSim/Quartus II Simulator. Somente após a validação individual de cada bloco os módulos foram integrados em um único arquivo de topo, `genius_top.sv`, sintetizado para a placa DE2 com entidade de topo `jogo`. Essa abordagem permitiu isolar erros de temporização e de lógica combinacional antes da integração final, reduzindo o retrabalho durante a etapa de síntese.

2.2. Arquitetura Geral do Sistema

O sistema foi particionado em seis submódulos, todos instanciados dentro do módulo de topo `genius_top`:

- **debounce**: filtro anti-ruído para os botões (`KEY[3:0]`) e para a chave de Início (`SW[15]`);
- **gerador_aleatorio**: gerador de cores pseudoaleatórias baseado em LFSR (*Linear Feedback Shift Register*);
- **memoria_sequencia**: memória que armazena a sequência de cores sorteadas ao longo da partida;
- **controlador_audio**: gera os sinais de áudio do bipe de jogo e da melodia de derrota, tocados no buzzer;
- **genius_fsm**: máquina de estados finitos que controla todo o fluxo do jogo;
- **pontuacao**: contador de pontos em BCD e decodificador para os displays de sete segmentos (`HEX0--HEX7`).

As chaves `SW[17]`, `SW[16]` e `SW[15]` da DE2 foram mapeadas, respectivamente, para Liga/Desliga, Reset e Início, conforme especificado no enunciado do projeto. Os botões `KEY[3:0]` foram mapeados para as cores Verde, Vermelho, Azul e Amarelo. As saídas de LED do jogo (`led_verde`, `led_vermelho`, `led_azul`, `led_amarelo`) e o sinal do buzzer foram roteadas para pinos do conector de expansão `GPIO_1`, aos quais foram conectados os quatro LEDs externos (com resistores de $470\ \Omega$) e o buzzer montados em protoboard, conforme sugerido no enunciado.

2.3. Filtro de Debounce

Como os botões e chaves mecânicos da DE2 produzem ruído de contato ao serem pressionados, foi implementado um módulo `debounce` para cada entrada crítica (as quatro cores e a chave de Início). O filtro sincroniza o sinal de entrada com dois flip-flops em série e só atualiza a saída depois que o valor sincronizado permanece estável durante 2^{20} ciclos de clock (aproximadamente $21\ ms$ a $50\ MHz$), eliminando transições espúrias.

2.4. Geração de Cores Aleatórias

A sequência de cores do jogo é sorteada por um LFSR de 4 bits (`gerador_aleatorio`), cujo polinômio de realimentação é dado por

$feedback = \sim(lfsr[3] \wedge lfsr[2])$. A cada ciclo de clock o registrador desloca seus bits e insere o valor de realimentação na posição menos significativa; os dois bits menos significativos do registrador ($lfsr[1:0]$) são usados como o índice da cor sorteada (00 = Amarelo, 01 = Azul, 10 = Vermelho, 11 = Verde). O registrador é inicializado com a semente $4'b0001$ sempre que o Reset geral é ativado, garantindo que o LFSR nunca fique preso no estado zero.

2.5. Memória da Sequência

O módulo `memoria_sequencia` armazena, em um vetor de 32 posições de 2 bits, todas as cores já sorteadas na rodada corrente (respeitando o limite de 31 sequências definido no enunciado). A cada novo acerto do jogador, uma nova cor gerada pelo LFSR é gravada na posição apontada por um ponteiro interno, que é incrementado a cada gravação e zerado sempre que ocorre um Reset ou o pulso de `limpa_mem`, emitido pela FSM ao entrar no estado `IDLE`. A leitura da memória é combinacional, endereçada pelo sinal `end_leitura`, permitindo que a FSM reproduza a sequência inteira, passo a passo, a cada rodada.

2.6. Máquina de Estados Finita Principal

O núcleo de controle do jogo é a máquina de estados `genius_fsm`, implementada com oito estados:

Tabela 1. Estados da máquina `genius_fsm`.

Estado	Descrição
IDLE	Estado de repouso/animação de "boas-vindas" nos LEDs; zera a pontuação atual e limpa a memória da sequência enquanto o jogo estiver desligado ou aguardando o Início.
GERA	Sorteia e grava uma nova cor na sequência por meio do LFSR.
MOSTRA	Reproduz, cor a cor, toda a sequência armazenada, acionando o LED correspondente e um bipe curto no buzzer.
ESPERA	Aguarda o jogador pressionar um botão; se nenhum botão for pressionado em até 5 segundos (<i>timeout</i> de 250×10^6 ciclos a 50 MHz), o jogo é considerado perdido.
VERIFICA	Compara a jogada do usuário com a cor esperada da sequência; em caso de acerto, incrementa a pontuação, em caso de erro, encaminha para o estado de derrota.
ESPERA_SOLTAR	Mantém o LED e o bipe ativos enquanto o botão está pressionado e aguarda a liberação do botão antes de avançar para o próximo passo da sequência.
PREPARA_PROX	Pequena pausa entre rodadas antes de sortear a próxima cor.
DERROTA	Aciona a melodia de derrota no buzzer, acende todos os LEDs e mantém a pontuação máxima atualizada; permanece neste estado até a chave de Início ser desligada.

A cada acerto do jogador na última cor da sequência, a FSM emite um pulso em `inc_ponto_n` (ativo baixo), que incrementa a pontuação em 10 pontos no módulo `pontuacao`, e retorna ao estado GERA para sortear uma nova cor e reapresentar a sequência completa, conforme especificado no enunciado. Ao ligar a chave de Início após uma pausa, a FSM retoma exatamente do estado em que parou, já que o registrador de estado só é reiniciado por Reset ou pela chave Liga/Desliga.

O trecho a seguir ilustra a lógica de transição do estado ESPERA, responsável pela condição de derrota por tempo excedido:

Listing 1. Transição do estado ESPERA (arquivo `genius.top.sv`).

```
1 ESPERA: begin
2     if (apertou) proximo = VERIFICA;
3     else if (timer >= 28'd250_000_000) proximo = DERROTA;
4 end
```

2.7. Módulo de Pontuação

O módulo `pontuacao` mantém dois contadores de três dígitos em BCD (pontuação atual e pontuação máxima), incrementados de 10 em 10 conforme exigido no enunciado. Um detector de borda de subida no sinal `key_inc` (proveniente da FSM) dispara o incremento da dezena; ao ultrapassar 9, há propagação de vai-um (*carry*) para a centena e, em seguida, para o milhar. A pontuação máxima é atualizada continuamente por comparação vetorial com a pontuação atual (`{milhar, centena, dezena}`), de modo que o recorde é preservado mesmo quando a pontuação atual é zerada por um pulso de Reset. Ao desligar a chave Liga/Desliga, ambos os contadores (atual e máximo) são zerados, e os displays são apagados, atendendo ao requisito de que "ao ser desligado, todos os LEDs e displays são desligados". Cada dígito BCD é convertido para o padrão de sete segmentos por meio do módulo `decoder7`, instanciado oito vezes (quatro dígitos para a pontuação atual em `HEX0--HEX3` e quatro para a pontuação máxima em `HEX4--HEX7`, sendo o dígito de unidade sempre fixado em zero, já que a pontuação varia de 10 em 10).

2.8. Controlador de Áudio

O módulo `controlador_audio` gera uma onda quadrada no `buzzer` por meio de um contador que divide a frequência de 50 MHz até a frequência desejada. Durante o estado `MOSTRA/ESPERA_SOLTAR`, é gerado um bipe curto de aproximadamente 440 Hz (nota musical A4). Ao entrar no estado `DERROTA`, o controlador reproduz uma pequena melodia de derrota, percorrendo uma tabela de oito notas (G4, E4, C4, C4, D4, E4, G4, G4) armazenadas em um vetor de constantes, cada uma soada por um intervalo fixo de tempo antes de avançar para a próxima nota da tabela. A intenção aqui era gerar o hino do Internacional FC, time de coração do professor Mandelli.

2.9. Mapeamento de Pinos e Síntese

A associação dos sinais lógicos aos pinos físicos da DE2 foi feita no *Pin Planner* do Quartus II e registrada no arquivo `jogo.qsf`. O resumo das atribuições utilizadas está na Tabela 2.

Tabela 2. Mapeamento de sinais para pinos da DE2.

Sinal do projeto	Pino/Recurso da DE2	Função
clk_50mhz	CLOCK_50	Clock principal (50 MHz)
sw_liga_desliga	SW[17]	Liga/Desliga geral
sw_reset_geral	SW[16]	Reset (pontuação atual)
sw_inicio_jogo	SW[15]	Início/pausa do jogo
key_verde, key_vermelho, key_azul, key_amarelo	KEY[3], KEY[2], KEY[1], KEY[0]	Botões de cor
hex0–hex7	HEX0–HEX7	Displays de 7 segmentos
led_verde, led_vermelho, led_azul, led_amarelo	GPIO_1[0–3]	LEDs externos (protoboard)
buzzer	GPIO_1[4]	Buzzer (protoboard)

Após a atribuição de pinos, o projeto foi compilado (*Analysis & Synthesis, Fitter e Assembler*) no Quartus II, gerando o arquivo de configuração `jogo.sof`, utilizado para programar a FPGA da DE2 por meio do *Programmer* em modo JTAG.

2.10. Simulação e Testes

Antes da integração final, os módulos de pontuação e de controle de LEDs/buzzer foram simulados separadamente por meio de formas de onda de teste (arquivos `.vwf`) no simulador do Quartus II/ModelSim, permitindo verificar visualmente a contagem em BCD, a atualização do recorde e a temporização do bipe e da melodia de derrota antes de serem integrados ao módulo de topo. Após a integração, o sistema completo foi testado diretamente na placa DE2, percorrendo manualmente os casos de uso descritos no enunciado: início e pausa do jogo pela chave de Início, incremento da pontuação a cada acerto, ativação da condição de derrota por erro de sequência, ativação da condição de derrota por estouro do tempo de 5 segundos, atualização da pontuação máxima e reinício de uma nova partida pelo botão de Reset sem perda do recorde. O funcionamento pode ser verificado por no link

3. Conclusão

O jogo Genius foi implementado em SystemVerilog seguindo a metodologia descrita na Seção 2.1, na qual cada submódulo – debounce, LFSR, memória da sequência, máquina de estados, pontuação e controlador de áudio – foi simulado isoladamente antes da integração no arquivo de topo e da síntese para a placa DE2. Essa abordagem cumpriu seu papel: os erros de temporização e de lógica combinacional foram isolados cedo, o que facilitou a integração final. Os testes na placa (Seção 2.10) confirmaram o funcionamento de todas as regras do enunciado – início/pausa do jogo, incremento de pontuação, derrota por erro de sequência e por estouro de tempo, e preservação do recorde após reset –.

4. Autoavaliação

O projeto foi dividido em 3 partes, a pontuação, a máquina de estados(que engloba todo o funcionamento e controle do jogo), e o som, cada um dos três integrantes contribui em todas as 3 partes. Os maiores desafios foram definir a lógica da máquina de estados e integralizar todas as três partes em um arquivo só (top level), pois mexer com HDL em projetos desse tipo não é tão simples.

Referências

[Floyd 2011] Floyd, T. L. (2011). *Sistemas Digitais: Fundamentos e Aplicações*. Bookman.

[Floyd 2011]